27

Controlling Image Display

So far, the examples in this book haven't rendered any images on mobile screens. This doesn't reflect the reality of mobile applications. Web applications display lots of images. If anything, native mobile applications rely on images even more than web applications because images are a powerful tool when you have a limited amount of space.

In this chapter, you'll learn how to use the React Native `Image` component, starting with loading images from different sources. Then, you'll learn how you can use the `Image` component to resize images, and how you can set placeholders for lazily loaded images. Finally, you'll learn how to implement icons using the `@expo/vector-icons` package. These sections cover the most common use cases for using images and icons in apps.

We'll cover the following topics in this chapter:

- Loading images

- Resizing images
- Lazy image loading
- Rendering icons

Technical requirements

You can find the code and image files for this chapter on GitHub at **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter27**.

Loading images

Let's get started by figuring out how to load images. You can render the `<Image>` component and pass its properties just like any other React component. But this particular component needs image blob data to be of any use. A **BLOB** (short for **Binary Large Object**) is a data type used to store large, un-structured binary data. BLOBs are commonly used to store multimedia files like images, audio, and video.

Let's look at some code:

```
const reactLogo = "https://reactnative.dev/docs/assets/favicon.png";
const relayLogo = require("./assets/relay.png");
export default function App() {
  return (
    <View style={styles.container}>
      <Image style={styles.image} source={{ uri: reactLogo }} />
```

```
        <Image style={styles.image} source={relayLogo} />
      </View>
    );
  }
```

There are two ways to load the blob data into an `<Image>` component. The first approach loads the image data from the network. This is done by passing an object with a **URI** property to the `source` code. The second `<Image>` component in this example is using a local image file. It does this by calling `require()` and passing the result to the `source` code.

Now, let's see what the rendered result looks like:

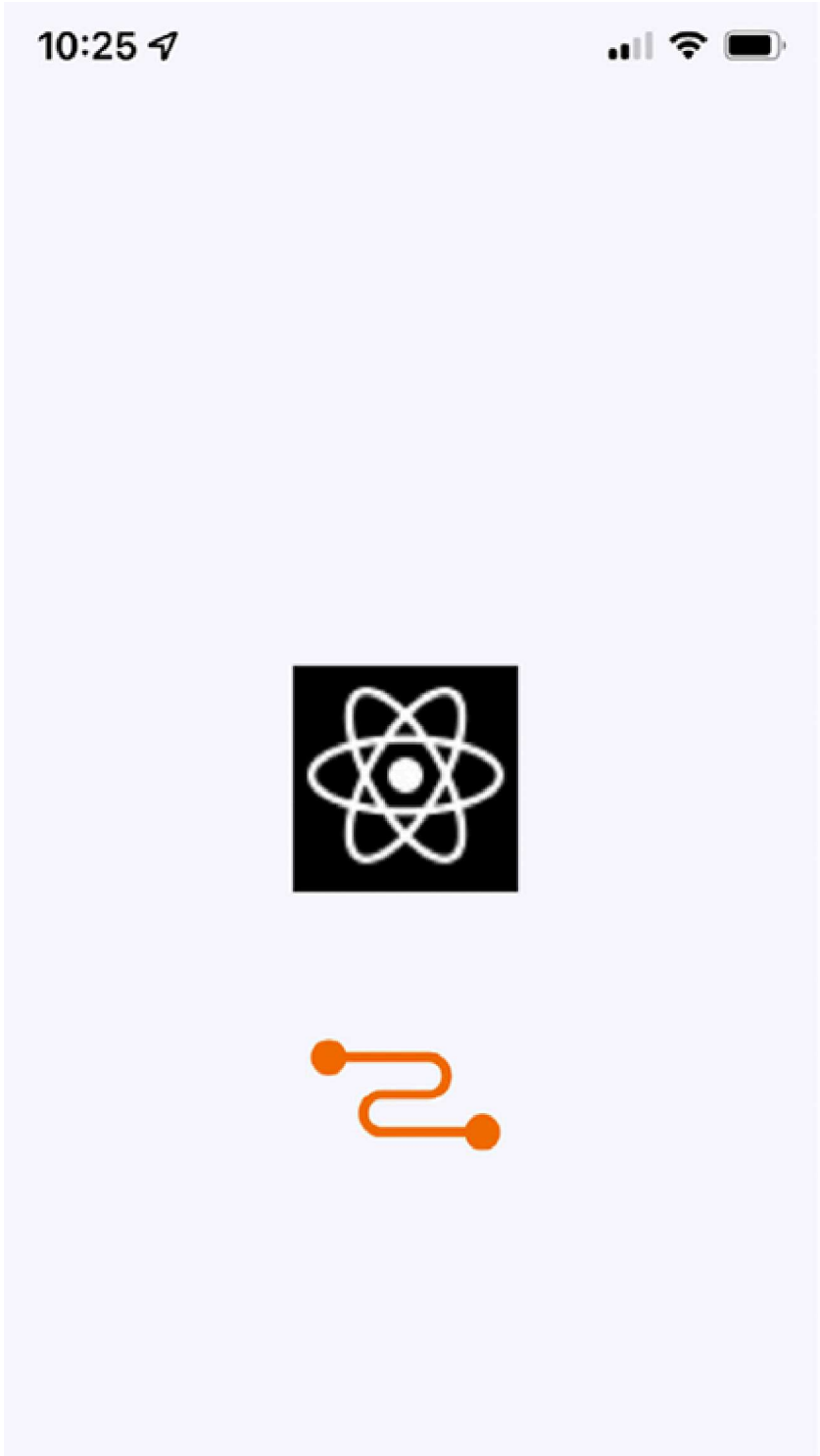

10:25

Figure 27.1: Image loading

Here's the style that was used with these images:

```
image: {
  width: 100,
  height: 100,
  margin: 20,
},
```

Note that without the `width` and `height` style properties, images will not render. In the next section, you'll learn how image resizing works when the `width` and `height` values are set.

Resizing images

The `width` and `height` style properties of `Image` components determine the size of what's rendered on the screen. For example, you'll probably have to work with images at some point that have a larger resolution than you want to be displayed in your React Native application. Simply setting the `width` and `height` style properties on the `Image` is enough to properly scale the image.

Let's look at some code that lets you dynamically adjust the dimensions of an image using controls:

```
export default function App() {
  const source = require("./assets/flux.png");
  const [width, setWidth] = useState(100);
  const [height, setHeight] = useState(100);
  return (
    <View style={styles.container}>
      <Image source={source} style={{ width, height }} />
      <Text>Width: {width}</Text>
      <Text>Height: {height}</Text>
      <Slider
        style={styles.slider}
        minimumValue={50}
        maximumValue={150}
        value={width}
        onValueChange={(value) => {
          setWidth(value);
          setHeight(value);
```

```
          }}
        />
      </View>
    );
  }
```

Here's what the image looks like if you're using the default 100 x 100 dimensions:

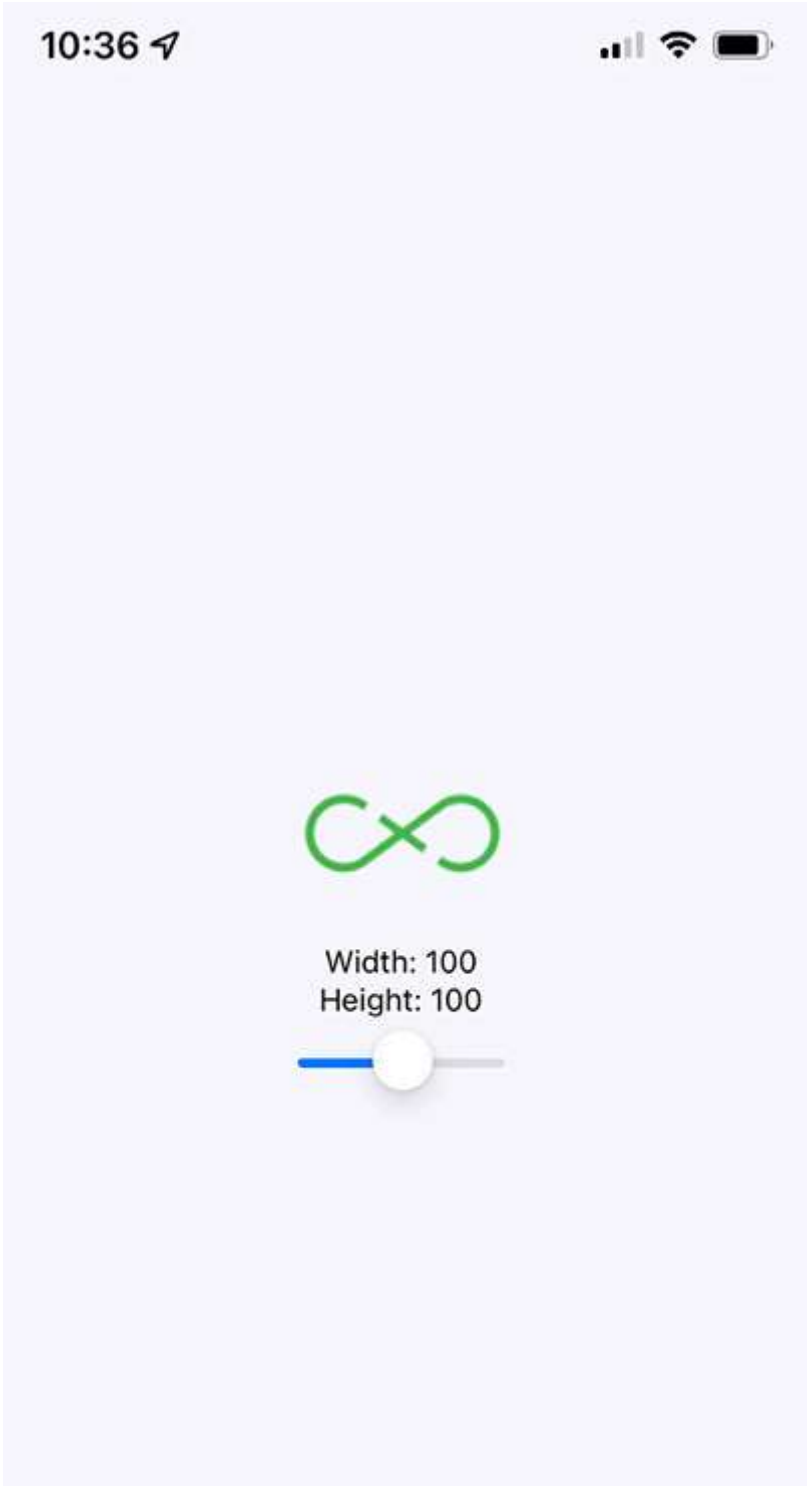

10:36
Width: 100
Height: 100

Figure 27.2: 100 x 100 image

Here’s a scaled-down version of the image:

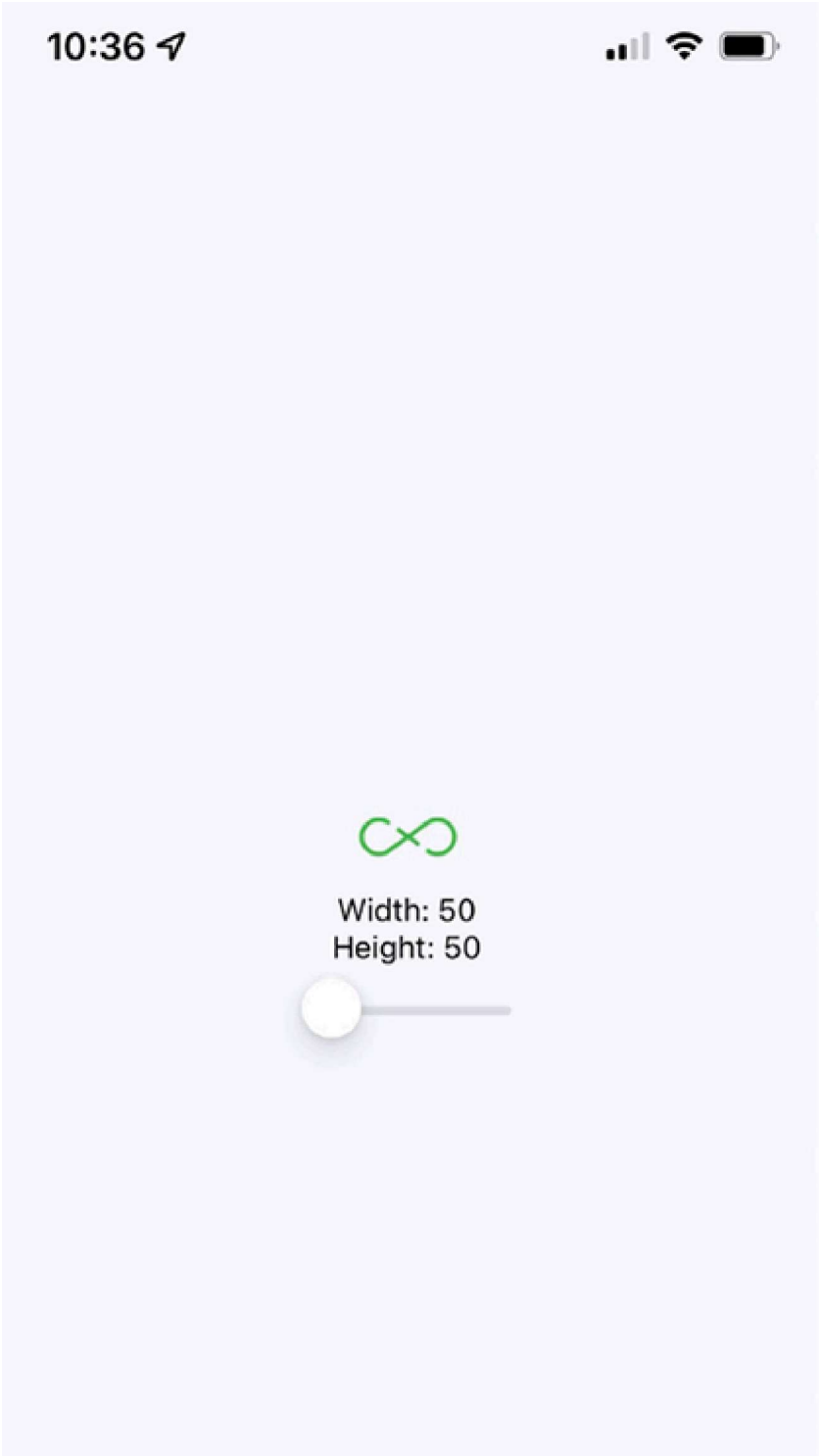

10:36
Width: 50
Height: 50

Figure 27.3: 50 x 50 image

Lastly, here's a scaled-up version of the image:

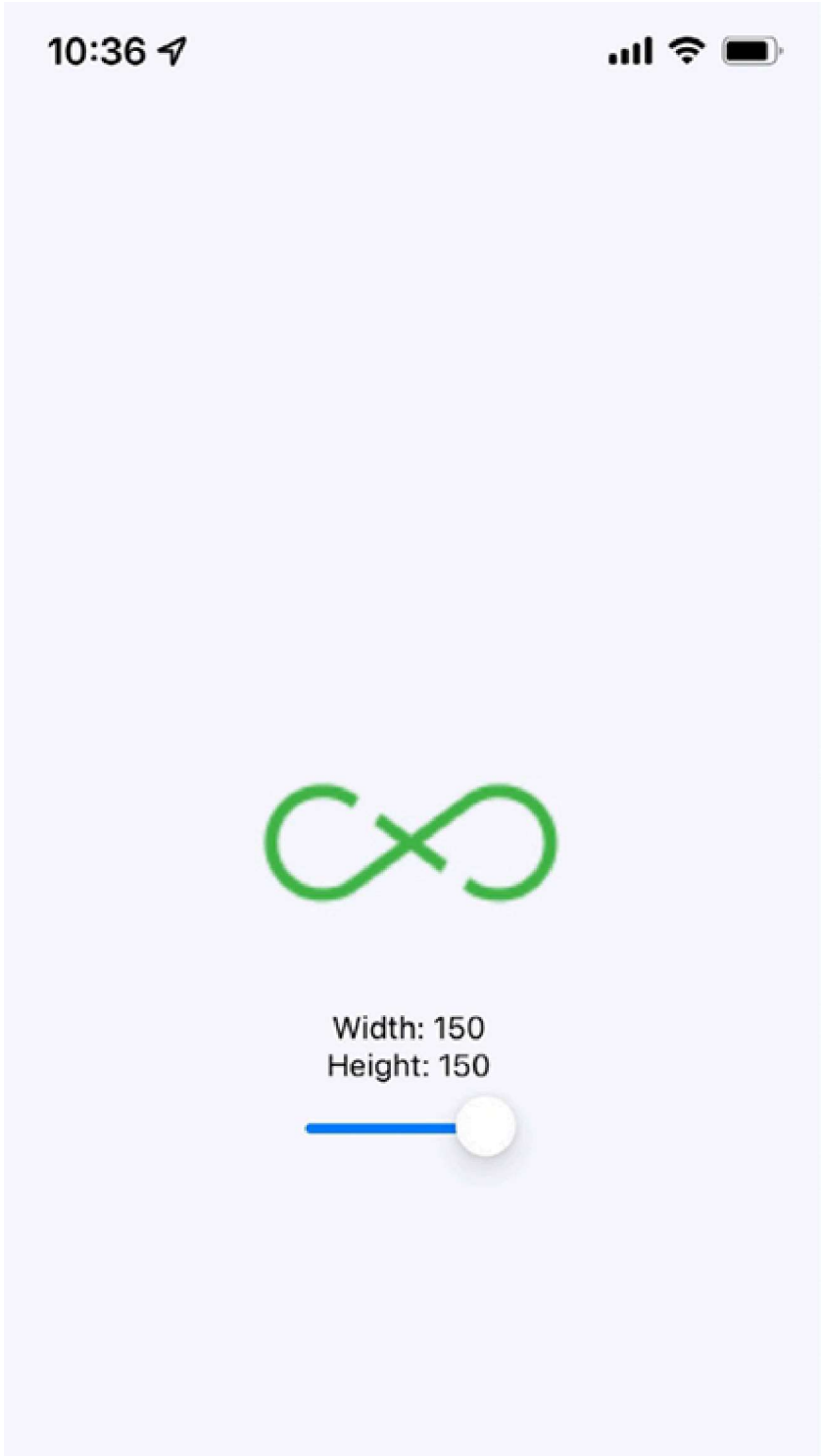

10:36
Width: 150
Height: 150

Figure 27.4: 150 x 150 image

There's a `resizeMode` property that you can pass to `Image` components. This determines how the scaled image fits within the dimensions of the actual component. You'll see this property in action in the *Rendering icons* section of this chapter.

As you can see, the dimensions of the images are controlled by the `width` and `height` style properties. Images can even be resized while the app is running by changing these values. In the next section, you'll learn how to lazily load images.

Lazy image loading

Sometimes, you don't necessarily want an image to load at the exact moment that it's rendered; for example, you might be

rendering something that's not visible on the screen yet. Most of the time, it's perfectly fine to fetch the image source from the network before it's actually visible. But if you're fine-tuning your application and discover that loading lots of images over the network causes performance issues, you can use the lazy loading strategy.

I think the more common use case in a mobile context is handling a scenario where you've rendered one or more images where they're visible, but the network is slow to respond. In this case, you will probably want to render a placeholder image so that the user sees something right away, rather than an empty space. So, let's get started.

Firstly, you can implement an abstraction that wraps the actual image that you want to show once it's loaded. Here's the code for this:

```
const placeholder = require("./assets/placeholder.png");
type PlaceholderProps = {
  loaded: boolean;
  style: StyleProp<ImageStyle>;
};
function Placeholder({ loaded, style }: PlaceholderProps) {
  if (loaded) {
    return null;
  } else {
```

```
      return <Image style={style} source={placeholder} />;
    }
  }
```

Now, here, you can see the placeholder image will be rendered only while the original image isn't loaded:

```
type Props = {
  style: StyleProp<ImageStyle>;
  resizeMode: ImageProps["resizeMode"];
  source: ImageSourcePropType | null;
};
export default function LazyImage({ style, resizeMode, source }: Props) {
  const [loaded, setLoaded] = useState(false);
  return (
    <View style={style}>
      {!!source ? (
        <Image
          source={source}
          resizeMode={resizeMode}
          style={style}
          onLoad={() => {
            setLoaded(true);
          }}
        />
      ) : (
        <Placeholder loaded={loaded} style={style} />
      )}
    </View>
```

```
    );
  }
```

This component renders a `View` component with two `Image` components inside it. It also has a loaded state, which is initially `false`. When `loaded` is `false`, the placeholder image is rendered. The `loaded` state is set to `true` when the `onLoad()` handler is called. This means that the placeholder image is removed and the main image is displayed.

Now, let's use the `LazyImage` component that we've just implemented. You'll render the image without a `source`, and the placeholder image should be displayed. Let's add a button that gives the lazy image a `source`. When it loads, the placeholder image should be replaced. Here's what the main `app` module looks like:

```
const remote = "https://reactnative.dev/docs/assets/favicon.png";
export default function LazyLoading() {
  const [source, setSource] = useState<ImageSourcePropType | null>(null);
  return (
    <View style={styles.container}>
      <LazyImage
        style={{ width: 200, height: 150 }}
        resizeMode="contain"
        source={source}
      />
```

```
        <Button
          label="Load Remote"
          onPress={() => {
            setSource({ uri: remote });
          }}
        />
      </View>
    );
  }
```

This is what the screen looks like initially:

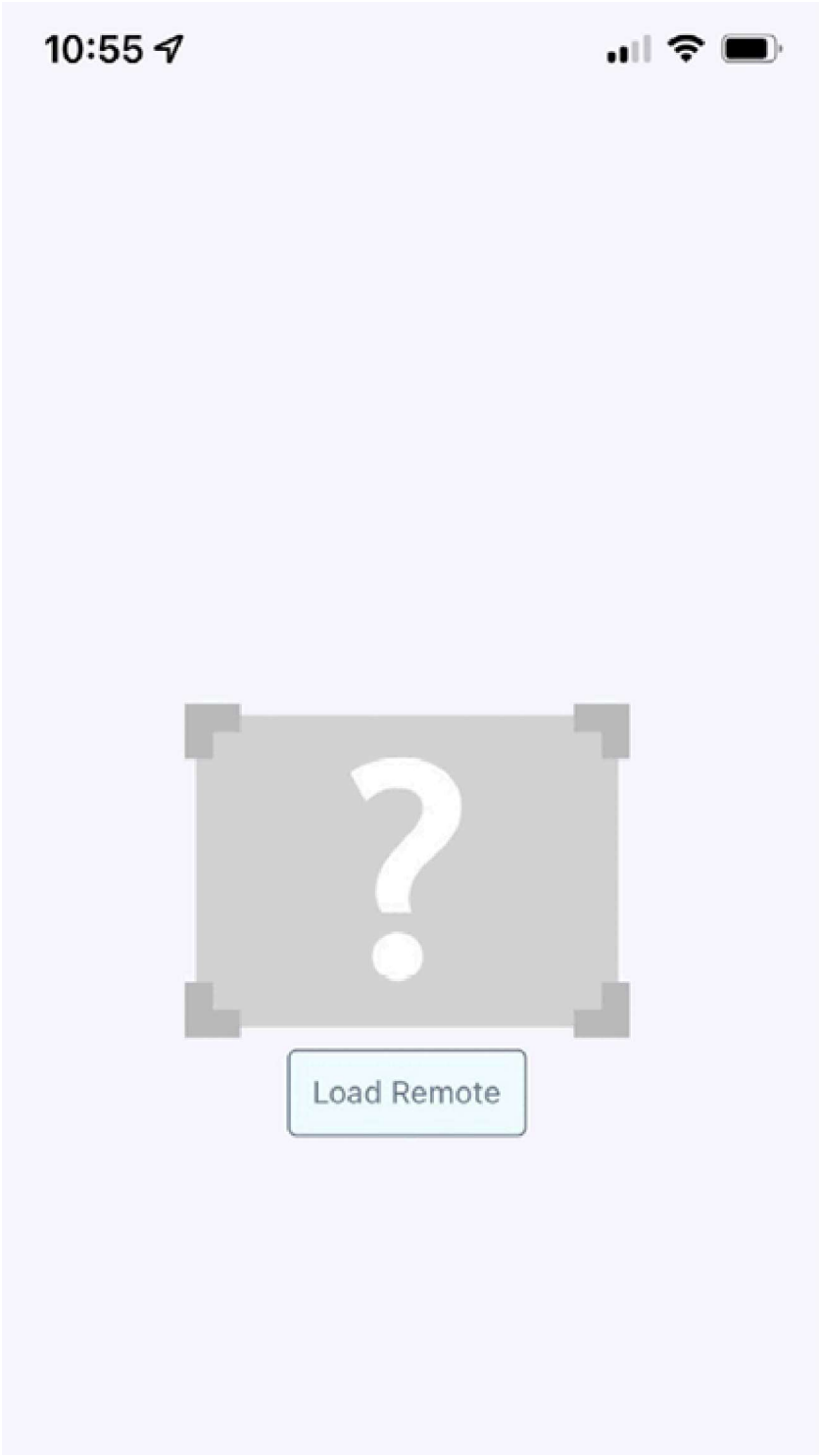

10:55
Load Remote

Figure 27.5: Initial state of the image

Then, click the **Load Remote** button to eventually see the image that we actually want:


10:55
Load Remote

Figure 27.6: Loaded image

You might notice that, depending on your network speed, the placeholder image remains visible, even after you click the **Load Remote** button. This is by design because you don't want to remove the placeholder image until you know for sure that the actual image is ready to be displayed. Now, let's render some icons in our React Native application.

Rendering icons

In the final section of this chapter, you'll learn how to render icons in React Native components. Using icons to indicate meaning makes web applications more usable. So, why should native mobile applications be any different?

We'll use the `@expo/vector-icons` package to pull various vector font packages into your React Native app. This package is already part of the Expo project that we're using as the base of the app, and now, you can import `Icon` components and render them. Let's implement an example that renders several **FontAwesome** icons based on a selected icon category:

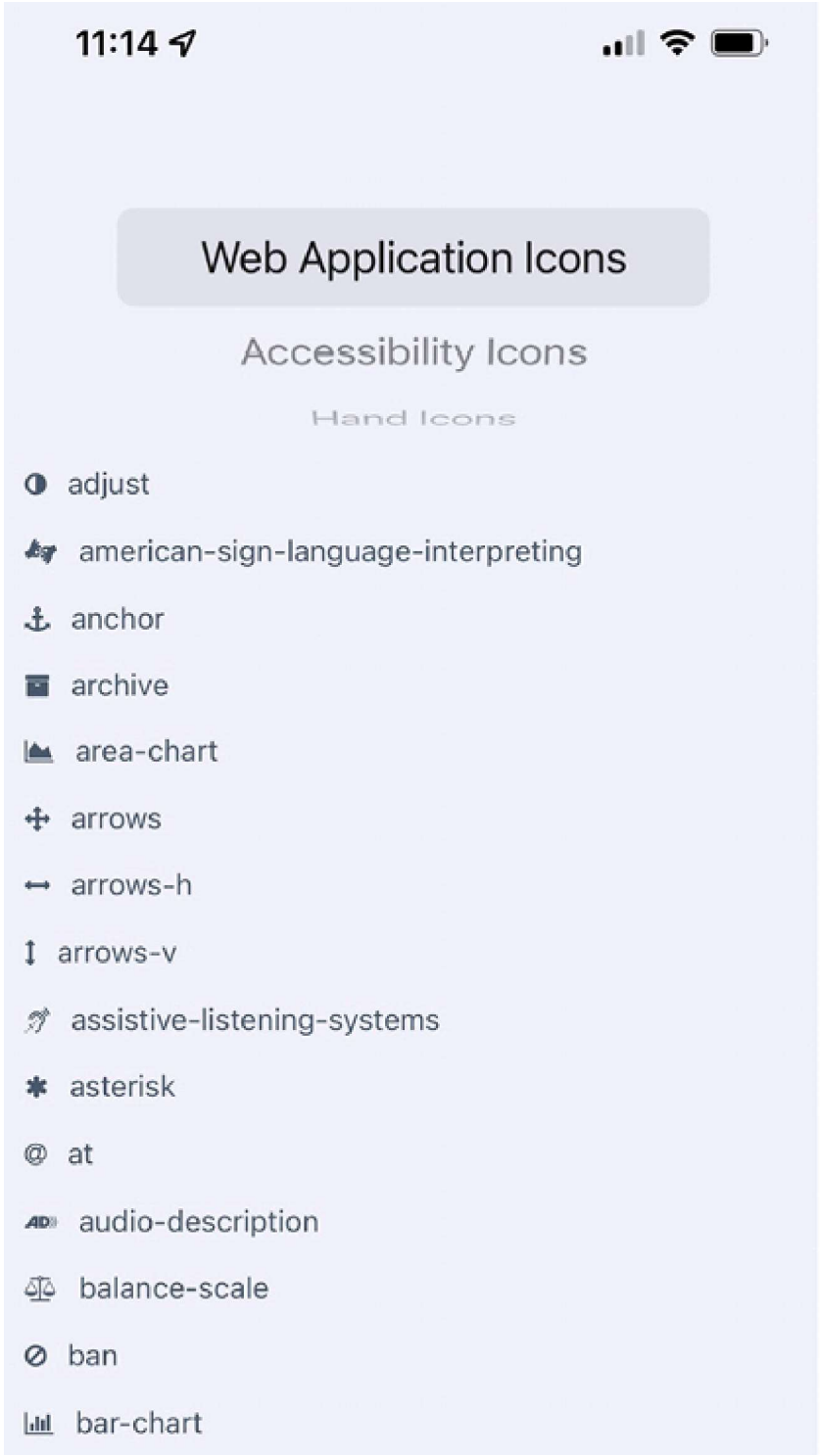
```
export default function RenderingIcons() {
  const [selected, setSelected] = useState<IconsType>("web_app_icons");
  const [listSource, setListSource] = useState<IconName[]>([]);
  const categories = Object.keys(iconNames);
  function updateListSource(selected: IconsType) {
    const listSource = iconNames[selected] as any;
    setListSource(listSource);
    setSelected(selected);
  }
  useEffect(() => {
    updateListSource(selected);
  }, []);
```

Here, we have defined all necessary logic to store and update the icon data. Next, we will apply it to the layout:

```
  return (
    <View style={styles.container}>
      <View style={styles.picker}>
        <Picker selectedValue={selected} onValueChange={updateListSource}>
          {categories.map((category) => (
```

```
              <Picker.Item key={category} label={category} value={category} />
            ))}
          </Picker>
        </View>
        <FlatList
          style={styles.icons}
          data={listSource.map((value, key) => ({ key: key.toString(), value }))}
          renderItem={({ item }) => (
            <View style={styles.item}>
              <Icon name={item.value} style={styles.itemIcon} />
              <Text style={styles.itemText}>{item.value}</Text>
            </View>
          )}
        />
      </View>
    );
  }
```

When you run this example, you should see something that looks like the following:


11:14
Web Application Icons
Accessibility Icons
Hand Icons
adjust
american-sign-language-interpreting
anchor
archive
area-chart
arrows
arrows-h
arrows-v
assistive-listening-systems
asterisk
at
audio-description
balance-scale
ban
bar-chart

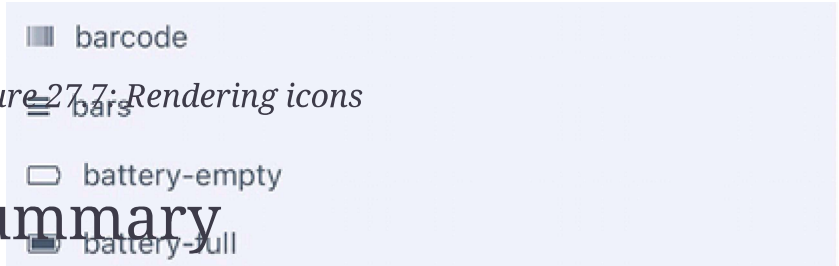


Figure 27.7: Rendering icons

Summary

In this chapter, we learned about handling images in our React Native applications. Images in a native application are just as important in a native mobile context as they are in a web context: they improve the user experience.

We learned about the different approaches to loading images, as well as how to resize them. We also learned how to implement a lazy image, which displays a placeholder image while the actual image is loading. Finally, we learned how to use icons in a React Native app. These skills will help you manage images and make your app more informative.

In the next chapter, we'll learn about local storage in React Native, which is handy when our app goes offline.